

SIH26183 — Master Project Report

Real-Time Identification of Fraud-Linked Cryptocurrency Exchanges from Victim-Reported Suspect Wallet Addresses

Problem Statement ID: SIH26183 **Theme:** Blockchain & Cybersecurity **Sponsoring Ministry:** Ministry of Home Affairs **Track:** Software **Team size:** 6

One-line pitch:

"From a victim's reported wallet address to a confidence-scored, evidence-backed cash-out report — in under a minute, not a manual multi-hour trace."

Table of Contents

1. [The Problem](#)
2. [Our Solution — How We Solve It](#)
3. [Current Solutions & How We Differ](#)
4. [Advantages](#)
5. [Disadvantages & Honest Limitations](#)
6. [The Hard Parts & How We Overcome Them](#)
7. [Feature Set — MVP vs Stretch](#)
8. [System Architecture](#)
9. [Frontend Screens](#)
10. [Backend API Design](#)
11. [Database Schema](#)
12. [Blockchain Data Pipeline — Real API Constraints](#)
13. [The Detection Engine — Pattern Library & Scoring](#)
14. [Attribution Data Strategy](#)

15. [Evaluation — Getting Real Numbers](#)
 16. [Team Division \(6 Members\)](#)
 17. [Build Timeline](#)
 18. [5-Minute Demo Script](#)
 19. [Judge Q&A Preparation](#)
 20. [Risk Register](#)
-

1. The Problem

1.1 The Situation

When an Indian citizen loses money to a cryptocurrency scam — a fake investment platform, a romance scam, a fraudulent trading app — they report it through the national cybercrime helpline (1930) or the cybercrime.gov.in portal. What the investigating officer receives is typically **one artifact: a suspect wallet address**.

From that single address, the officer needs to answer a question that determines whether the money is recoverable at all:

Where did the money go, and which exchange did it get cashed out through?

This matters because a blockchain address is pseudonymous and unfreezable — but an **exchange account** is not. Exchanges have KYC records and can freeze balances. Identifying the cash-out exchange, fast, is the only realistic path to freezing funds and identifying a suspect.

1.2 Why It's Hard Today

Barrier	Impact
Manual tracing	An officer using free block explorers must click through transactions hop by hop. A scammer who splits funds across 300+ addresses makes this practically impossible by hand.
Speed	Funds move to an exchange and get withdrawn in hours. A trace that takes days is a trace that arrives after the money is gone.
Skill scarcity	Blockchain forensics is a specialist skill. Most district-level cybercrime cells have no trained on-chain investigator.

Cost of tooling	Professional platforms exist, but at enterprise licensing costs that put them out of reach for local cells and smaller units.
Deliberate obfuscation	Scammers use layering, peel chains, and mixing-like patterns specifically designed to defeat manual tracing.

1.3 What the Problem Statement Asks For

An automated system that ingests a victim-reported suspect wallet address and identifies, in real time, the fraud-linked exchange(s) through which the funds moved — output in a form an investigator can actually act on.

2. Our Solution — How We Solve It

2.1 The Reframe

Most teams will read this PS and build “a graph explorer.” That’s the wrong product. The correct product is:

A cybercrime investigator’s forensic assistant — a guided investigation workflow that goes from complaint intake to a signed, evidence-backed report.

The pipeline we build:

```

Victim complaint
  ↓
Suspect wallet address
  ↓
Automated multi-hop transaction graph
  ↓
Entity clustering (collapse 300 addresses → 6 actors)
  ↓
Obfuscation / laundering pattern detection
  ↓
Exchange attribution + confidence scoring
  ↓
Human review (confirm / reject / annotate)
  ↓
Tamper-evident investigator report (PDF + evidence hash)

```

2.2 Step-by-Step Method

Step 1 — Case Intake The investigator enters the suspect wallet address, chain (BTC/ETH),

and a case/complaint reference ID. The system opens a case record — this is not a stateless lookup tool, it's a case-based workflow that mirrors how a real investigation is filed.

Step 2 — Transaction Graph Construction Pull the address's full public transaction history via blockchain APIs and expand outward N hops, building a directed graph. Edges are weighted by:

- **Amount** — a ₹8 lakh transfer matters more than a ₹200 dust transaction
- **Time-decay** — recent movement is more actionable than movement from two years ago
- **Hop depth** — confidence decays with distance from the seed address

Dust filtering and depth caps prevent combinatorial explosion.

Step 3 — Entity Clustering Apply the standard heuristics real forensic investigators use:

- **Common-input-ownership heuristic** — in a UTXO chain like Bitcoin, if multiple addresses are spent together as inputs to one transaction, they are almost always controlled by the same entity. This is the single most powerful clustering tool in blockchain forensics.
- **Change-address detection** — identify which output of a transaction is the sender's own change, letting us follow the actor rather than the payment.

The effect: a visually overwhelming 300-address graph collapses into a comprehensible 5–8 **actor clusters**.

Note: on account-based chains like Ethereum, common-input-ownership doesn't apply the same way, so for the EVM adapter we lean on behavioral signals — funding source, timing correlation, contract interaction patterns, and known service labels — which is exactly how professional analysts handle the account model.

Step 4 — Obfuscation & Laundering Pattern Detection (*our core technical layer*) Rather than dumping a graph and asking a human to spot the pattern, we run an explicit detection engine over the graph structure looking for the specific *shapes* laundering makes: rapid fan-out, fan-in consolidation, peel chains, round-number splitting, and timing bursts. (Full library in [Section 13](#).)

Step 5 — Exchange Attribution Match terminal clusters against a curated attribution dataset of known exchange deposit addresses, mixer addresses, and sanctioned addresses, producing a ranked list of probable cash-out points with confidence scores. (Data sources in [Section 14](#).)

Step 6 — Human-in-the-Loop Review Every flagged finding is presented for the investigator to **Confirm / Reject / Annotate**. The system never asserts guilt — it says “*this*

wallet exhibits patterns consistent with layering; further investigation recommended.” This is both ethically correct and legally necessary.

Step 7 — Evidence-Grade Report Generate a one-page PDF containing case metadata, financial summary, the evidence trail (transaction hashes, timestamps, amounts, detected patterns), the confidence-scored conclusion, and a **SHA-256 hash of the evidence payload** so the report can be verified as unmodified later.

2.3 Design Principles

Every conclusion the system produces must answer five questions:

Question	System must show
WHAT happened?	The traced fund movement
WHY is it suspicious?	The specific patterns detected, with weights
WHERE did the money go?	The attributed exit cluster
WHEN did it happen?	Timestamps on every hop
HOW CONFIDENT are we?	An explicit score with a visible breakdown

Confidence, never certainty. Attribution data is incomplete by nature; the UI language reflects this everywhere.

3. Current Solutions & How We Differ

3.1 The Existing Landscape

Category	Tools	What they do well	Gap for our use case
Enterprise forensics	Chainalysis (Reactor / KYT), TRM Labs, Elliptic Investigator, Crystal Intelligence	Industry-standard graph tracing, entity clustering, real-time risk scoring, huge proprietary attribution datasets. Chainalysis is used by 150+ public sector organizations across 40+ countries, and TRM maintains probabilistic entity profiles even	Enterprise licensing costs; closed-source black-box scoring; not accessible to a district cybercrime cell or

		across chain-hops and swaps	self-hostable
Law-enforcement-focused	Merkle Science Tracker	Built specifically for law enforcement workflows including transaction visualization, clustering, and risk scoring	Still a paid enterprise product; not customizable to India's specific complaint-intake workflow
Free OSINT explorers	Etherscan, Blockchair, Tronscan, Arkham, Breadcrumbs, MetaSleuth	Free, public, and widely used — skilled investigators combine multiple tools rather than relying on one platform	Manual, hop-by-hop. No automated clustering, no obfuscation detection, no confidence scoring, no report generation. All cognitive load sits on the officer.
Open-source analytics	GraphSense (open forensics platform), BlockSci	Genuinely open-source, and provides a curated collection of over 500,000 publicly available attribution tags	A research/analytics platform, not an investigator-facing product; no guided case workflow, no report output
Emerging ML approaches	Elliptic's graph-neural-network direction	Analyzes wallet behavior texture rather than following purely linear paths, targeting dormant "sleeper clusters"	Proprietary; also not something a hackathon team can or should try to replicate

3.2 The Actual Gap

The market splits cleanly into two halves:

- **Powerful but inaccessible** (Chainalysis, TRM, Elliptic, Merkle Science) — enterprise-priced, closed, general-purpose compliance tooling.
- **Accessible but manual** (Etherscan, Blockchair, Breadcrumbs) — free, but they hand

you raw data and expect you to be an expert.

Nothing occupies the middle for Indian law enforcement: free/self-hostable, automated end-to-end, and shaped around the specific workflow of *a citizen complaint arriving at 1930 with one wallet address attached*.

3.3 Our Differentiation — Stated Precisely

We are not trying to out-build Chainalysis. We are building the free, self-hostable, India-workflow-first slice of what they do: automated clustering + explainable obfuscation detection + a confidence-scored cash-out report, generated from a single victim-reported address.

Four specific differentiators:

1. **Workflow-native, not tool-native.** Case IDs, complaint references, confirm/reject review, exportable evidence reports. It fits into a case file, not a browser tab.
2. **Explainable by construction.** Our scoring is an inspectable rule set, not a black box. An investigator (or a defence lawyer, or a judge) can interrogate exactly why any score was assigned. Commercial platforms' risk scores range from rule-based systems with explicit auditable logic to machine learning models — we deliberately choose the auditable end, because in a legal-evidence context that's a feature, not a limitation.
3. **Evidence integrity built in.** Cryptographic hashing of every evidence payload — turning output from "a screenshot" into "a verifiable artifact."
4. **Honest confidence.** The system is designed to say "trail degrades here" rather than fabricate a conclusion. Analytics tools can produce false certainty, and address labeling and clustering always require careful interpretation — we build that caution into the UI language.

4. Advantages

4.1 Technical Advantages

Advantage	Detail
Real, live public data	Bitcoin and Ethereum ledgers are fully public. The demo runs on a genuine trace of a real address — no mock JSON, which judges can smell instantly.
No GPU/heavy	This is a graph + rules problem. It runs on a laptop. No training

ML dependency	infrastructure, no model convergence risk mid-hackathon.
Deterministic and reproducible	Same input → same output, every time. Critical for a legal-evidence context and for a demo that can't afford to be flaky.
Genuinely extensible architecture	Chain-agnostic adapter design means new chains are additions, not rewrites.
Open attribution data exists	GraphSense TagPacks is a curated collection of attribution tags collected from public sources, each required to carry a dereferenceable link to its source and to exclude personally identifiable information — legally clean, citable, and free.

4.2 Strategic / Competition Advantages

Advantage	Detail
Low expected competition	The PS sounds intimidating ("blockchain forensics"), so most teams self-select toward safer, more familiar topics. Fewer teams to be compared against.
Strong demo visual	An animated fund-flow graph collapsing 300 addresses into a flagged exchange is immediately comprehensible to a non-technical judge.
Named national workflow	"1930 helpline → wallet address → investigator" is a specific, real, government-owned process. Judges reward solutions that slot into an existing pipeline.
Sponsoring-ministry fit	MHA owns cybercrime. The output is exactly what MHA's cells need. Direct alignment.
Defensible under questioning	Because everything is rule-based and explainable, there's no question a judge can ask that ends in "we're not sure why the model did that."

4.3 Societal Value

- Reduces investigation time from hours/days of manual tracing to minutes.
- Democratizes a specialist capability to cells that can't afford enterprise tooling.
- Improves the odds of actual fund recovery, since exchange freezes are time-sensitive.

- Produces court-usable, tamper-evident documentation.

5. Disadvantages & Honest Limitations

These must be stated in the pitch, not hidden. A team that names its own limits reads as credible; a team that gets caught hiding one reads as amateur.

5.1 Technical Limitations

Limitation	Reality	Our stance
Cross-chain laundering breaks the trail	Once funds cross into a privacy coin or a cross-chain bridge, deterministic on-chain tracing effectively ends. This is true even for the best commercial tools — TRM maintains entity profiles across such hops probabilistically, not with certainty.	We detect and <i>flag</i> the bridge/swap hop as a confidence boundary, and report “trail confidence degrades here” rather than guessing.
Attribution data is incomplete	Open tag datasets don’t cover every exchange, especially smaller regional or offshore ones. Some traces will honestly dead-end.	Report “unattributed exit cluster” explicitly. An honest dead-end is more useful to an investigator than a false confident answer.
Clustering heuristics have known failure modes	CoinJoin-style activity deliberately breaks the common-input-ownership assumption, and the change-address heuristic can misfire on certain wallet software.	Detect CoinJoin-like structures and reduce cluster confidence rather than reporting a false merge.
API rate limits constrain graph depth	Free-tier blockchain APIs cap how fast we can expand a graph (see Section 12).	Aggressive caching + configurable depth caps. Named as a deployment consideration, not hidden.
No ground-truth “criminal”	We cannot prove any wallet belongs to a criminal. We can only show behavioral patterns.	This is precisely why the tool is decision-support with mandatory human review.

5.2 Product / Deployment Limitations

- **Requires an investigator to interpret output.** It's an assistant, not an autonomous system. That's by design, but it means adoption needs some training.
- **Curated tag list needs maintenance.** Attribution data goes stale as exchanges rotate addresses. A real deployment needs a data-refresh process.
- **Not a fund-recovery tool.** Blockchain investigations don't recover funds directly — they produce the documented evidence that enables recovery through other channels. We must not overclaim here.

5.3 Ethical & Legal Considerations

Concern	Mitigation
Could be framed as a surveillance/deanonymization tool	Scoped strictly to victim-initiated complaints with a case ID. Not a general-population scanner. State this explicitly and early.
Risk of false accusation	Language is always probabilistic ("patterns consistent with..."), never accusatory. Mandatory human confirm/reject on every finding.
Chain of custody for evidence	Addressed via evidence hashing and immutable audit records.
Privacy of innocent third parties	Clustering can sweep in unrelated addresses. Clusters are labeled by confidence, and low-confidence members are visually de-emphasized rather than asserted.

5.4 Presentation Risk

The topic can read as **dry or overly technical** if the pitch opens with graph theory. The mitigation is narrative sequencing — open with the human story (a citizen lost ₹8.4 lakh), then show the machine. See [Section 18](#).

6. The Hard Parts & How We Overcome Them

Hard Part #1 — Obfuscation Detection (the real technical core)

Why it's hard: Anyone can write a graph traversal in a day. The genuinely difficult problem is distinguishing *"this is a legitimate exchange consolidating customer deposits"* from *"this is a scammer layering funds to break the trail."* Both look like fan-in/fan-out on a graph. This distinction is the entire value of the product — get it wrong and you're a slow Etherscan clone.

How we overcome it:

1. **Start with forensic-literature heuristics, not ML.** Implement the well-documented laundering shapes (peel chain, rapid fan-out, round-number splitting, timing burst) as explicit rules. These are fast to build, fast to explain, and immediately beat a raw graph dump.
 2. **Use contextual disambiguation.** A legitimate exchange cluster has high transaction volume, long operational history, and appears in attribution data. A laundering cluster is newly created, has burst activity, then goes dormant. Encode these as discriminating features — this is what separates a naive fan-out detector from a useful one.
 3. **Add a lightweight supervised layer only if time allows.** Gradient-boosted trees on hand-engineered graph features (fan-out ratio, hop timing variance, cluster age, amount entropy). **Not** a from-scratch GNN.
 4. **Keep the GNN as a "future work" slide.** Judges respect a working heuristic system far more than a broken "AI" one. Say plainly: *"We chose explainable rules over a black-box model because this output goes into a legal case file."*
-

Hard Part #2 — Attribution Without Enterprise Data

Why it's hard: Chainalysis's moat is a proprietary attribution database built over a decade. We cannot replicate that in a hackathon, and exit-point identification is the actual deliverable of the PS.

How we overcome it:

1. **Use the open datasets that exist.** GraphSense's public TagPacks carry exchange hot wallets, ransomware endpoints, scam clusters, and sanctioned addresses, under mostly permissive open licenses (MIT / CC0), with the upstream pack name retained so license obligations stay traceable.
2. **Layer in the official sanctions list.** OFAC publishes sanctioned crypto addresses from the US Treasury as a public, auto-updated source — free and authoritative.
3. **Manually curate a small high-value list.** For Indian-relevant exchanges, hand-verify a modest set of known deposit addresses from public sources. 50 well-verified tags beat

50,000 unverified ones for demo credibility.

4. **Be transparent about coverage.** The UI should show *“matched against N attribution sources covering M known entities”* rather than implying omniscience.
-

Hard Part #3 — Live API Reliability During the Demo

Why it’s hard: Free-tier blockchain APIs are rate-limited (see Section 12). Expanding a multi-hop graph across hundreds of addresses can exhaust a quota in seconds. **The single most common way this project fails is a rate-limit error appearing on screen in front of judges.**

How we overcome it:

1. **Build a caching layer in hour one.** Every API response cached locally. Graph expansion re-queries the same addresses constantly; without a cache you will burn your quota.
 2. **Pre-fetch and cache the demo wallets days in advance.** The scripted demo runs entirely from cache.
 3. **Run a live fetch only during Q&A**, framed deliberately: *“That was cached for demo speed — here, let me pull a fresh address live for you.”* This turns a defensive measure into a credibility moment.
 4. **Get API keys approved in advance.** Blockchair’s free non-commercial key requires an email application. Do not leave this to the night before.
-

Hard Part #4 — Proving It Works (Validation)

Why it’s hard: There’s no public labeled dataset of “scam wallet → correct exchange” pairs. It’s tempting to invent precision/recall numbers. **Do not.** Fabricated metrics are the fastest way to lose a technically competent judge, and they’re indefensible in Q&A.

How we overcome it:

1. **Build a small labeled set from publicly documented cases.** Historical exchange hacks, ransomware campaigns, and mixer cases where researchers or news outlets already published the laundering path. 15–30 cases is enough. Note that GraphSense TagPacks includes packs specifically for hacks and ransomware — a useful starting point for identifying documented cases.
2. **Report only what you measured.** *“Correctly flagged 11 of 14 known laundering patterns, 2 false positives”* is more credible than a polished-looking 92%/91% table you

can't defend.

3. **Use the time comparison — it's unfakeable.** Manually trace one case yourself with a stopwatch, then run your system on it. "45 minutes manual vs. 3 minutes automated, same case" is a legitimate, self-generated, honest metric.
4. **Show a failure case on purpose.** Demo one trace that dead-ends at a bridge, and show the system correctly reporting degraded confidence. This is a *strength* signal — it proves the system doesn't hallucinate answers.

Hard Part #5 — Making a Graph Comprehensible in 90 Seconds

Why it's hard: A 300-node force-directed graph is visual noise. Judges have ~5 minutes and are not blockchain analysts.

How we overcome it:

1. **Animate the collapse.** Show 327 raw addresses → animate the clustering → 6 clean actor nodes. The *transition* is the story; a static "after" image isn't.
2. **Progressive disclosure.** Default view shows clusters only. Detail is one click away, not on screen by default.
3. **Colour-code by meaning**, not aesthetics: victim (neutral) → suspect (red) → intermediary (amber) → exchange (green/attributed).
4. **One narrative path.** Highlight the single highest-confidence money trail and dim everything else. Show one story, not all possible stories.

7. Feature Set — MVP vs Stretch

7.1 MVP — Must Work Live

#	Feature	Why it's MVP
1	Case intake (wallet + chain + case ID)	Establishes the investigator-workflow identity
2	Multi-hop transaction graph from live chain data	Core data layer
	Entity clustering (common-input + change-	

3	address)	The “300 → 6” collapse; core value
4	Pattern detection engine (5 patterns)	The actual technical differentiator
5	Explainable confidence score with breakdown	The “Why?” panel — key credibility feature
6	Exchange attribution against curated tag set	Directly answers the PS question
7	Follow-the-Money animated graph	The demo centerpiece
8	PDF report with SHA-256 evidence hash	Makes it deployable, not just demoable
9	Confirm / Reject / Note on findings	Human-in-the-loop; ethical necessity

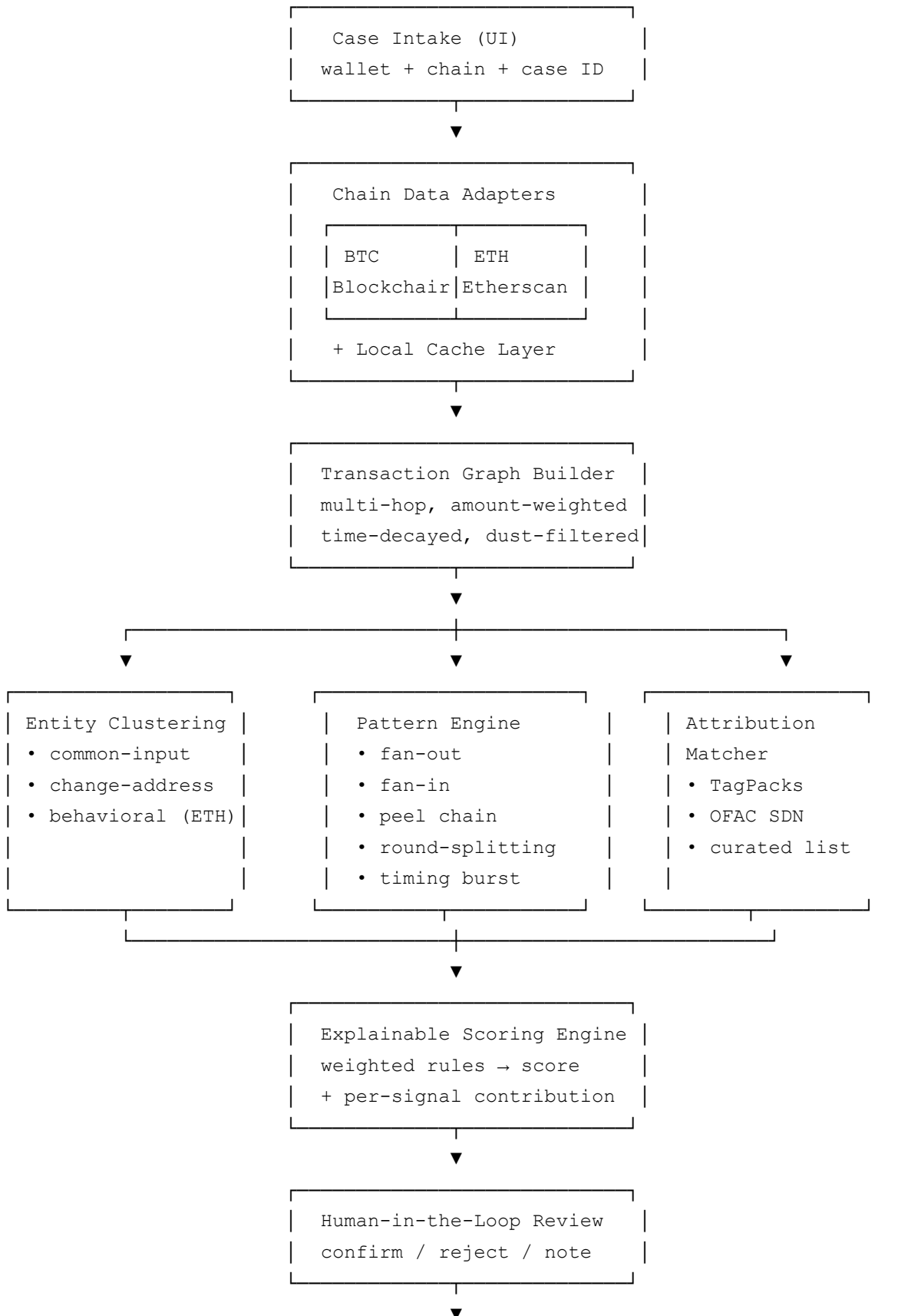
7.2 Stretch — Only If MVP Is Solid With 6+ Hours Left

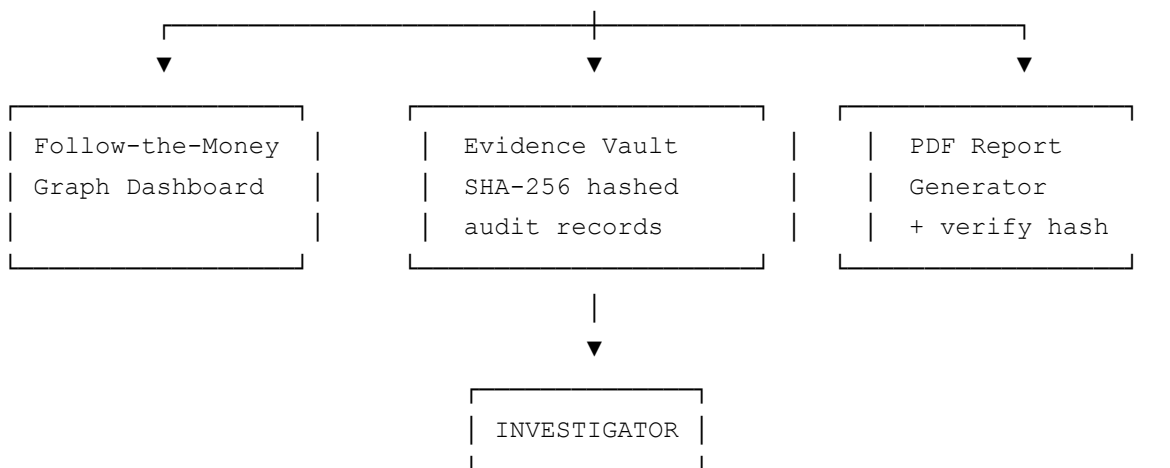
Feature	Value
Chronological “Investigation Replay” animation	High demo impact, zero functional necessity
Case management list view (multiple saved cases)	Deployment realism
Second-chain hardening (polish whichever adapter is weaker)	Robustness
Lightweight supervised classifier layer	Technical depth signal

7.3 Explicitly Out of Scope — And Why

Not building	Reason
15-chain support	Shallow breadth reads as padding. BTC + ETH built properly, adapter architecture stated honestly.
From-scratch GNN	High risk, low explainability, wrong tool for a legal-evidence context.
Automated exchange reporting/freezing	Legally inappropriate for an automated system to trigger.
Real-time monitoring of all chain activity	Scope explosion; also crosses into surveillance framing.

8. System Architecture





8.1 Recommended Stack

Layer	Choice	Rationale
Backend	Python + FastAPI	Graph manipulation is fastest in Python (networkx); FastAPI gives auto-generated API docs which is a free credibility point in the demo
Graph engine	networkx	Mature, well-documented, no infra needed
Frontend	React + Vite + Tailwind	Fast iteration; team likely already comfortable
Graph visualization	A force-directed graph library (react-force-graph / D3)	Animation support is essential for the centerpiece demo
Database	PostgreSQL (or SQLite for zero-setup demo)	SQLite is the safer hackathon choice — one file, no service to crash on stage
Cache	Local SQLite table or Redis	Non-negotiable; see Section 12
PDF generation	Server-side PDF library	Keep it simple; a clean one-page layout beats a fancy multi-page one

Deliberate non-requirement: no GPU, no model training, no cloud dependency. The entire system runs on one laptop, offline, from cache. This is a robustness advantage worth stating.

9. Frontend Screens

#	Screen	Purpose	Key elements
1	Case Intake	Start an investigation	Wallet address field, chain selector, case/complaint ID, large "INVESTIGATE" action
2	Investigation Dashboard	At-a-glance case summary	Risk level, amount traced, wallets discovered, transactions analysed, clusters found, obfuscation detected Y/N
3	Follow-the-Money Graph	The centerpiece	Animated hop-by-hop flow, cluster collapse animation, colour-coded node roles, highlighted primary trail
4	Why? Panel	Explainability	Per-signal score contributions, with the actual transaction hashes backing each flag
5	Findings Review	Human-in-the-loop	List of flagged patterns, each with Confirm / Reject / Add Note
6	Attribution Result	Answers the PS question	Ranked probable exit points, confidence per candidate, attribution source cited per match
7	Report Preview & Export	Deliverable	One-page report preview, "Generate PDF", displayed evidence hash, verify-hash action

9.1 Investigation Dashboard — Layout Sketch

CASE: CYB-2026-00124		Chain: Ethereum	[REPORT ►]
RISK LEVEL: HIGH		OBFUSCATION DETECTED: YES	
Wallets discovered [live]	Transactions analysed [live]	Clusters identified [live]	
MOST LIKELY CASH-OUT POINT			

▶ [Attributed entity]	Confidence: [computed]	
Source: [attribution dataset cited]	[WHY?]	
DETECTED PATTERNS		
• Rapid fan-out	[Confirm] [Reject] [Note]	
• Peel chain	[Confirm] [Reject] [Note]	
• Timing burst	[Confirm] [Reject] [Note]	

Build note: every bracketed value above is computed at runtime. Do **not** hard-code display numbers into the UI, even as placeholders that might survive to demo day.

10. Backend API Design

POST	/api/cases	→ create case {complaint_id, wallet_address, chain}
POST	/api/cases/{id}/investigate	→ run the full pipeline; returns job/status for progress streaming
GET	/api/cases/{id}	→ case summary (counts, risk level, status)
GET	/api/cases/{id}/graph	→ nodes + edges for visualization (supports ?collapsed=true false)
GET	/api/cases/{id}/clusters	→ entity clusters with member addresses and confidence
GET	/api/cases/{id}/findings	→ detected patterns with score contributions and evidence tx hashes
POST	/api/cases/{id}/findings/{fid}/review	→ {action: confirm reject, note: string}
GET	/api/cases/{id}/attribution	→ ranked probable exit points with sources and confidence
POST	/api/cases/{id}/report	→ generate PDF; returns path + evidence hash
GET	/api/cases/{id}/evidence/{eid}/verify	→ recompute hash, return match true/false

```
GET      /api/health
        → API + cache + upstream adapter status
```

Progress streaming: `/investigate` should stream progress events (hop 1 complete, hop 2 complete, clustering, scoring) so the frontend can animate the graph building live. This is what makes the demo feel alive rather than showing a spinner for 20 seconds.

11. Database Schema

```
cases (
  id          UUID PRIMARY KEY,
  complaint_id TEXT,                -- e.g. CYB-2026-00124
  wallet_address TEXT NOT NULL,
  chain       TEXT NOT NULL,       -- 'bitcoin' | 'ethereum'
  status      TEXT,                -- pending|running|complete|failed
  risk_level  TEXT,
  created_at  TIMESTAMP,
  completed_at TIMESTAMP
)

wallets (
  id          UUID PRIMARY KEY,
  case_id     UUID REFERENCES cases(id),
  address     TEXT NOT NULL,
  hop_depth  INT,
  first_seen  TIMESTAMP,
  last_seen   TIMESTAMP,
  total_in   NUMERIC,
  total_out  NUMERIC,
  cluster_id  UUID REFERENCES clusters(id)
)

transactions (
  id          UUID PRIMARY KEY,
  case_id     UUID REFERENCES cases(id),
  tx_hash     TEXT NOT NULL,
  from_address TEXT,
  to_address  TEXT,
  amount      NUMERIC,
  amount_inr  NUMERIC,            -- converted at tx-time rate
  timestamp   TIMESTAMP,
  chain       TEXT,
  hop_depth  INT
```

```

)

clusters (
    id                UUID PRIMARY KEY,
    case_id           UUID REFERENCES cases(id),
    cluster_label     TEXT,
    member_count      INT,
    clustering_method TEXT,                -- common_input | change_address | behavior
    cluster_confidence NUMERIC,
    attributed_entity TEXT,                -- matched exchange/service, nullable
    attribution_source TEXT                -- tagpack | ofac | manual_curated
)

findings (
    id                UUID PRIMARY KEY,
    case_id           UUID REFERENCES cases(id),
    pattern_type      TEXT,                -- fan_out|fan_in|peel_chain|round_split|t
    score_contribution NUMERIC,
    evidence_tx_hashes TEXT[],
    detected_at       TIMESTAMP,
    status            TEXT,                -- flagged|confirmed|rejected
    analyst_note      TEXT,
    reviewed_by       TEXT,
    reviewed_at       TIMESTAMP
)

attribution_tags (
    address           TEXT PRIMARY KEY,
    entity_name       TEXT,
    entity_type       TEXT,                -- exchange|mixer|sanctioned|ransomware|se
    source            TEXT,                -- graphsense_tagpack | ofac_sdn | manual
    source_url        TEXT,                -- dereferenceable provenance link
    confidence        NUMERIC,
    last_updated      TIMESTAMP
)

evidence_records (
    id                UUID PRIMARY KEY,
    case_id           UUID REFERENCES cases(id),
    payload_json      JSONB,
    payload_hash      TEXT NOT NULL,      -- SHA-256
    created_at        TIMESTAMP
)

reports (
    id                UUID PRIMARY KEY,
    case_id           UUID REFERENCES cases(id),

```

```

pdf_path          TEXT,
evidence_hash     TEXT,
generated_at      TIMESTAMP,
generated_by      TEXT
)

api_cache (
  cache_key        TEXT PRIMARY KEY, -- chain:endpoint:address
  response_json    JSONB,
  fetched_at       TIMESTAMP,
  ttl_seconds      INT
)

```

Note on `attribution_tags.source_url` : keeping a provenance link per tag mirrors the GraphSense TagPack requirement that attribution tags must originate from public sources and must provide a dereferenceable pointer to the source. This is what lets you defend any attribution claim in Q&A — and in a real case file.

12. Blockchain Data Pipeline — Real API Constraints

12.1 Bitcoin — Blockchain

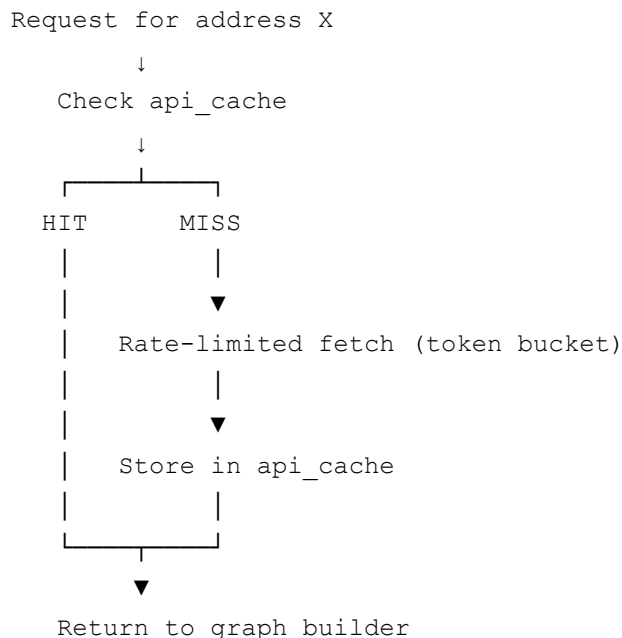
Property	Value
Free tier	API testing tier capped at 1,000 calls per day without an API key
Hard rate limit	30 requests/min on the free plan
Soft limit	5 requests/second on both free and paid plans, applied only under very high server load and toggled manually by admins
Free key path	Non-commercial and academic projects can apply for a free Public API key by email
Overage behaviour	Going over the free limit can result in the IP address being blocked from using the free API
Bulk option	Data Dumps provide bulk exports of blockchain data in TSV format, aimed at academic research and large-scale studies

12.2 Ethereum — Etherscan (V2)

Property	Value
Free tier	5 calls/second, up to 100,000 calls/day
Historical endpoints	Historical endpoints and name tags are capped at 2 calls/s regardless of tier
Key scope	A single free-tier key works across all Etherscan V2 chains, but the 5 calls/second limit is shared across all of them
Recent tightening	Etherscan's July 2026 changelog cut free-tier limits further — max records per request dropped from 10,000 to 1,000, and the Internal Transactions by Block Range endpoint is being removed from the free plan. Free API access was also restricted on several major networks including Avalanche, Base, BNB Chain and Optimism.
Under load	Rate limits are enforced strictly and return 429s under load

Implication: verify current free-tier terms before the event — they've been moving. Consider Blockscout as a fallback adapter, since its free tier covers all 120+ supported chains and migration is typically just a base URL and key swap while keeping existing endpoint structure and request logic intact.

12.3 Caching Strategy — Mandatory



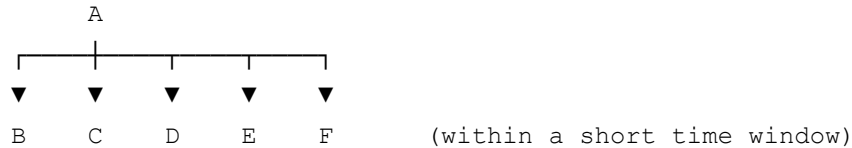
Rules:

1. Every upstream response is cached before use. No exceptions.
 2. Graph expansion uses a **token-bucket rate limiter** matched to the provider's documented limit (30/min for Blockchair, 5/sec for Etherscan) — never fire requests uncapped.
 3. Pre-fetch demo wallets **days before** the event; the scripted demo runs entirely from cache.
 4. Depth cap (default 4–5 hops) and dust filtering, both configurable, to prevent combinatorial blowup.
 5. Degrade gracefully: if a rate limit is hit, return the partial graph with a clear "expansion limited" notice — never crash, never show a raw 429 on screen.
-

13. The Detection Engine — Pattern Library & Scoring

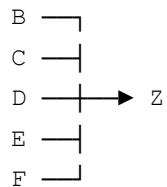
13.1 Pattern Library

Pattern 1 — Rapid Fan-Out



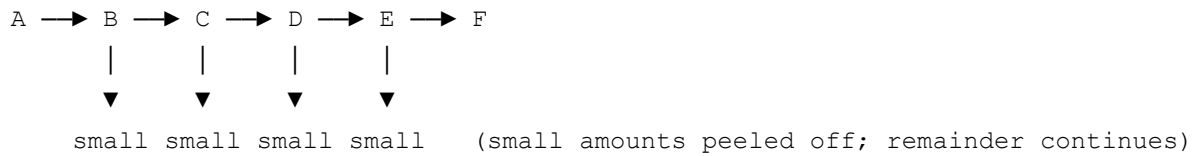
Detection: one address sends to $N \geq 5$ previously-unseen addresses within a configurable time window. *Meaning:* classic first step of layering — splitting funds to fragment the trail.

Pattern 2 — Fan-In / Consolidation



Detection: $N \geq 5$ addresses converge on a single address within a short window. *Meaning:* re-aggregation before cash-out. **Critical caveat:** legitimate exchange deposit consolidation looks identical — this is where the contextual disambiguation features (cluster age, attribution match, volume history) do the real work.

Pattern 3 — Peel Chain



Detection: a chain where each hop forwards a large remainder and peels a small amount to a side address. **Meaning:** one of the most recognizable deliberate laundering structures.

Pattern 4 — Round-Number Splitting

```

₹10,00,000
  ↓
₹2L + ₹2L + ₹2L + ₹2L + ₹2L

```

Detection: an amount split into near-equal or round-number pieces. **Meaning:** automated/scripted splitting rather than organic transaction behaviour.

Pattern 5 — Timing Burst

```

10:01:04  ┌
10:01:11  │
10:02:33  └─ multiple related txs in a tight window
10:02:47  │
10:03:02  └

```

Detection: transactions from related addresses clustered in an abnormally tight time window. **Meaning:** scripted movement, characteristic of automated laundering rather than human activity.

13.2 Scoring Model

Signal	Suggested weight	Rationale
Rapid fan-out	+20	Strong layering indicator
Peel chain	+20	Highly specific to deliberate obfuscation
Known mixer/sanctioned match	+20	Direct attribution hit
Fan-in consolidation	+15	Suggestive, but high false-positive rate alone

Round-number splitting	+15	Indicates scripting
Timing burst	+12	Supporting signal
Cluster newly created + burst + dormant	+15	Disambiguates laundering from legitimate services

Score presentation — the “Why?” panel:

SUSPICION SCORE: [computed]

Rapid fan-out	+ [w]	► 4 evidence txs
Peel chain detected	+ [w]	► 6 evidence txs
Round-number splitting	+ [w]	► 5 evidence txs
Timing burst	+ [w]	► 9 evidence txs
Total	[sum]	

13.3 Honesty Requirements — Non-Negotiable

1. **Weights are a documented starting heuristic**, tuned by hand and open to revision. Do **not** claim they were learned from a dataset unless they were.
2. **Every displayed number is computed at runtime** from the actual case. No hard-coded percentages anywhere in the UI.
3. **Call it “confidence,” never “certainty.”** Every conclusion is probabilistic.
4. **The scorecard is a transparent rule system, not “explainable AI.”** Describing it accurately is stronger than dressing it up — in Q&A, “it’s a rule engine and here’s every rule” is unassailable; “it’s explainable AI” invites a question you may not want.

14. Attribution Data Strategy

Source	What it gives	Licensing	How we use it
GraphSense public TagPacks	Exchange hot wallets, ransomware endpoints, scam clusters, sanctioned addresses; over 500,000 publicly available attribution tags	Mostly permissive open (MIT / CC0)	Primary bulk attribution layer; retain pack name per row for provenance
OFAC SDN	Officially sanctioned addresses across	Public	High-confidence

crypto addresses	BTC, ETH, TRX, XMR, USDT and others, from the US Treasury	government data	hard flags
Manually curated India-relevant list	Hand-verified deposit addresses for exchanges relevant to Indian cases	Self-collected from public sources	Small, high-precision layer for demo credibility

Ingestion note: TagPacks are YAML-structured with per-tag provenance. Load them into `attribution_tags` preserving `source` and `source_url`, so every attribution claim in a generated report is traceable back to its origin. In Q&A, “here’s the exact public source for that label” is the answer that ends the question.

Coverage honesty: the UI should always display “*matched against N sources covering M known entities*” — never imply exhaustive coverage.

15. Evaluation — Getting Real Numbers

15.1 Build a Small Labeled Test Set

Target: 15–30 cases. Sources of legitimate ground truth:

- Publicly documented historical exchange hacks where the laundering path was later published by researchers or journalists
- Ransomware campaigns with published payment-address analysis
- The `hacks` and `ransomware` packs within GraphSense TagPacks

For each case, record: seed address, known laundering pattern present, known eventual exit point (where published).

15.2 Metrics to Actually Measure

Metric	How to compute	Example honest phrasing
Pattern detection accuracy	Run pipeline, compare flags to known patterns	“Correctly flagged 11 of 14 known laundering patterns”
False positives	Count flags on legitimate/benign traces	“2 false positives across the test set”

Attribution hit rate	Cases where the correct exit was in our top-3 candidates	"Correct exit in top-3 for 9 of 12 attributable cases"
Time-to-trace	Stopwatch a manual trace vs. the system, same case	"45 min manual vs. 3 min automated"
Graph scale	Addresses/txs processed per run	Report actual live figures

15.3 The Rules

1. **Report only measured numbers.** If you didn't compute it, it doesn't go on a slide.
2. **The time comparison is your safest strong metric** — self-generated, unfakeable, and immediately intuitive to a judge.
3. **Show a failure case deliberately.** A trace that dead-ends at a bridge, with the system correctly reporting degraded confidence, proves the system doesn't fabricate answers. This lands better than a perfect record.
4. **Do this before hour 30.** Evaluation left to the last two hours doesn't happen.

16. Team Division (6 Members)

#	Role	Owns	Primary deliverables
1	Graph & Chain Engine Lead	Chain adapters, cache layer, rate limiter, graph builder	Working multi-hop graph from a live address by hour 8
2	Clustering Engineer	Common-input, change-address, behavioral clustering; cluster confidence	The "327 → 6" collapse working correctly
3	Pattern & Scoring Engineer	Pattern library, scoring engine, evaluation test set	5 patterns detecting + the Why? breakdown data
4	Frontend / Visualization Lead	Dashboard, Follow-the-Money graph, animations	The centerpiece visual, animating smoothly
5	Frontend / Workflow + Report	Intake, findings review UI, report preview, PDF + evidence hashing	End-to-end case flow, exportable PDF

6	Data, Attribution & Pitch Lead	TagPack/OFAC ingestion, curated list, API keys, demo script, slides, Q&A prep	Attribution DB populated; rehearsed 5-min pitch
---	---	---	---

16.1 Cross-Cutting Rules

- **Integration checkpoint every 6 hours.** Everyone merges and the full pipeline runs end-to-end, however ugly. A system integrated at hour 30 for the first time will not work.
- **Member 6 owns the demo, not just slides.** Someone must be accountable for the presentation from hour zero, or it becomes a panicked hour-34 activity.
- **Contract-first between backend and frontend.** Agree the JSON shapes in hour one so both sides build against a fixed interface and can work against mocks in parallel.
- **The pitch is rehearsed at least three times** end-to-end, out loud, on the actual demo machine.

17. Build Timeline

17.1 Pre-Hackathon (Critical — Do Not Skip)

Task	Owner	Why it can't wait
Apply for Blockchair free non-commercial API key	#6	Email approval takes time
Register Etherscan API key + verify current free-tier terms	#6	Terms have been tightening through 2026
Select and validate 2–3 real demo wallet addresses	#3, #6	Need one clean trace + one with a genuine documented laundering pattern
Build and test the cache layer offline	#1	The single biggest demo-failure risk
Ingest GraphSense TagPacks + OFAC list into <code>attribution_tags</code>	#6	Bulk data work; wasteful to do during the build window
Assemble the labeled evaluation set	#3	Research-heavy; will not happen under time pressure
Agree API contract (JSON shapes)	#1, #4	Unblocks parallel work from hour zero

17.2 Hackathon Window

Hours	Focus	Exit criterion
0–8	Chain adapters + graph builder + basic clustering, wired end-to-end (ugly is fine)	A real address produces a real multi-hop graph in the DB
8–16	Pattern engine + scoring, connected to the graph	Patterns detected, scores computed with per-signal breakdown
16–24	Frontend dashboard + Follow-the-Money visualization	The graph renders and animates from real backend data
24–30	Attribution matching, report generation, evidence hashing, confirm/reject UI	Full flow: address in → PDF out
30–34	Run the evaluation set, capture real numbers, polish demo	Slides carry only measured figures
34–36	Rehearse end-to-end x3, bug buffer, cache pre-warm	Pitch runs clean on the demo machine

Hard rule: feature freeze at hour 30. Anything not working by then is cut, not fixed. Stretch features are only opened after MVP passes an end-to-end run.

18. 5-Minute Demo Script

Structure principle: one compelling investigation, told as a story. Not a feature tour.

Time	Beat	What's on screen	What you say
0:00–0:30	The human story	Case intake screen	"A citizen reports a crypto scam to the 1930 helpline. They've lost their savings. All the investigator has is one wallet address. Today, tracing that manually takes hours — and the money is gone in hours."
0:30–1:00	The action	Paste address →	"One address. One click."

		INVESTIGATE	
1:00– 1:45	The scale reveal	Graph builds live, counters ticking	"The system has pulled and analysed [live count] transactions across [live count] wallets — work that would take an officer a full day by hand."
1:45– 2:15	The collapse	300+ nodes animate into ~6 clusters	"Using the same common-input-ownership heuristic professional forensic analysts use, those hundreds of addresses collapse into a handful of actual actors."
2:15– 2:45	The detection	Pattern flags light up	"And here's what the system found: rapid fan-out, a peel chain, round-number splitting. These are the specific shapes deliberate laundering makes."
2:45– 3:15	The explanation	Click → Why? panel	"We don't ask you to trust a score. Every point is attributed to a signal, and every signal links to the actual transaction hashes behind it. This is a rule engine — we can show you every rule."
3:15– 3:35	The answer	Attribution result	"Most likely cash-out point: [entity], confidence [computed], matched against [source]. That's the actionable output — because an exchange account, unlike a wallet, can be frozen."
3:35– 3:55	The deliverable	Generate PDF, show evidence hash	"One click produces a case-file-ready report with a cryptographic hash — so this evidence can be verified as unmodified later."
3:55– 4:25	The proof	Evaluation slide	"We validated against [N] publicly documented cases: [measured results]. And on the same case, manual tracing took us [X] minutes; this took [Y]."
4:25– 5:00	The honesty close	Limitations slide	"Two things we won't overclaim. Cross-chain and mixer hops degrade the trail — the system says so rather than guessing. And this never accuses anyone: every finding requires investigator confirmation. It's a forensic assistant, not a verdict machine."

18.1 Demo Rules

1. **Run from cache.** Pre-warmed, tested, offline-capable. Offer a live fetch during Q&A as

a credibility moment.

2. **Never show a raw error.** Every failure path degrades gracefully with a readable message.
3. **No hard-coded numbers on screen.** If a judge asks you to run a different address, everything must still be real.
4. **One person drives, one person narrates.** Splitting these prevents the “silent clicking” failure mode.
5. **Have a backup video** of a clean run on the machine, in case the live demo dies. Never use it unless you must.

19. Judge Q&A Preparation

Likely question	Prepared answer
<i>“How is this different from Chainalysis?”</i>	“We’re not competing with Chainalysis — we’re building the free, self-hostable slice a district cybercrime cell can actually deploy, shaped around India’s complaint-intake workflow. Different market, different constraint.”
<i>“Where does your attribution data come from?”</i>	“Open public sources — GraphSense TagPacks under permissive licenses, the OFAC sanctions list, plus a hand-curated set. Every tag retains a provenance link, so any attribution in the report is traceable to its source.”
<i>“What if the scammer uses a mixer or bridges to another chain?”</i>	“Then the deterministic trail ends, and the system says so — it flags a confidence boundary rather than guessing. Even the best commercial tools handle that probabilistically, not with certainty. Reporting an honest dead-end is more useful to an investigator than a confident wrong answer.”
<i>“Is this AI?”</i>	“The detection engine is a transparent rule system, deliberately. This output goes into a legal case file — an investigator, and eventually a court, needs to interrogate exactly why a conclusion was reached. A black-box model would be a liability here, not an upgrade.”
<i>“How do you know it works?”</i>	“We tested against [N] publicly documented laundering cases and measured [results]. We can also show you the time comparison — manual versus automated on the same case.”

<i>"Isn't this a surveillance tool?"</i>	"No — it only operates on a specific victim-initiated complaint with a case ID attached. It doesn't scan the chain broadly, and it never makes an assertion without investigator confirmation."
<i>"Can it scale to a real deployment?"</i>	"The bottleneck is upstream API rate limits, not our architecture. A real deployment would run against a self-hosted node or a bulk data dump rather than free-tier APIs — that's a documented, solved path."
<i>"What's next if you had more time?"</i>	"Three things: a supervised layer on top of the heuristics trained on a larger labeled corpus; broader chain adapters; and a shared attribution database that participating cells could contribute verified tags back into."

20. Risk Register

Risk	Likelihood	Impact	Mitigation
API rate limit hit during demo	High	Critical	Cache-first architecture; pre-warmed demo cache; token-bucket limiter; graceful degradation
Etherscan free-tier terms change again	Medium	High	Verify terms pre-event; keep Blockscout as a drop-in fallback adapter
Graph visualization is illegible	Medium	High	Cluster-collapse by default; progressive disclosure; highlight one primary trail
Clustering produces obviously wrong merges	Medium	Medium	CoinJoin detection reduces cluster confidence rather than asserting a false merge
Team integrates too late	Medium	Critical	Mandatory end-to-end integration checkpoint every 6 hours
Evaluation not done, tempting to fabricate	Medium	Critical	Test set built pre-hackathon; hard rule that unmeasured numbers never appear on a slide
Demo machine fails	Low	Critical	Backup machine with cache replicated; backup video as last resort
Pitch under-rehearsed	Medium	High	Member #6 owns it from hour zero;

Appendix A — One-Page Summary

Problem: A crypto scam victim reports one wallet address. Manually tracing where that money went — and which exchange it was cashed out through — takes hours of specialist work that most cybercrime cells can't do, while the funds disappear in hours.

Solution: An investigator's forensic assistant that takes the address, automatically builds and clusters the transaction graph, detects laundering patterns with an explainable rule engine, attributes the likely cash-out exchange with a confidence score, and produces a tamper-evident PDF report — with human confirmation required on every finding.

Differentiation: Enterprise tools are powerful but inaccessible; free explorers are accessible but entirely manual. Nothing occupies the middle for Indian law enforcement. We build that: free, self-hostable, workflow-native, explainable, honest about its limits.

Hardest problem: Distinguishing deliberate laundering from legitimate exchange activity — solved with forensic-literature heuristics plus contextual disambiguation features, deliberately chosen over a black-box model because this output goes into a legal case file.

Proof: Validated against publicly documented historical laundering cases, with measured detection results and a stopwatch comparison of manual versus automated tracing time.